

ID: 12511006

Name: Chandan Karmaker

Section: A

1)

a) A SQL CREATE TABLE statement for a student table using appropriate PRIMARY KEY, NOT NULL, UNIQUE and CHECK constraints is given below:

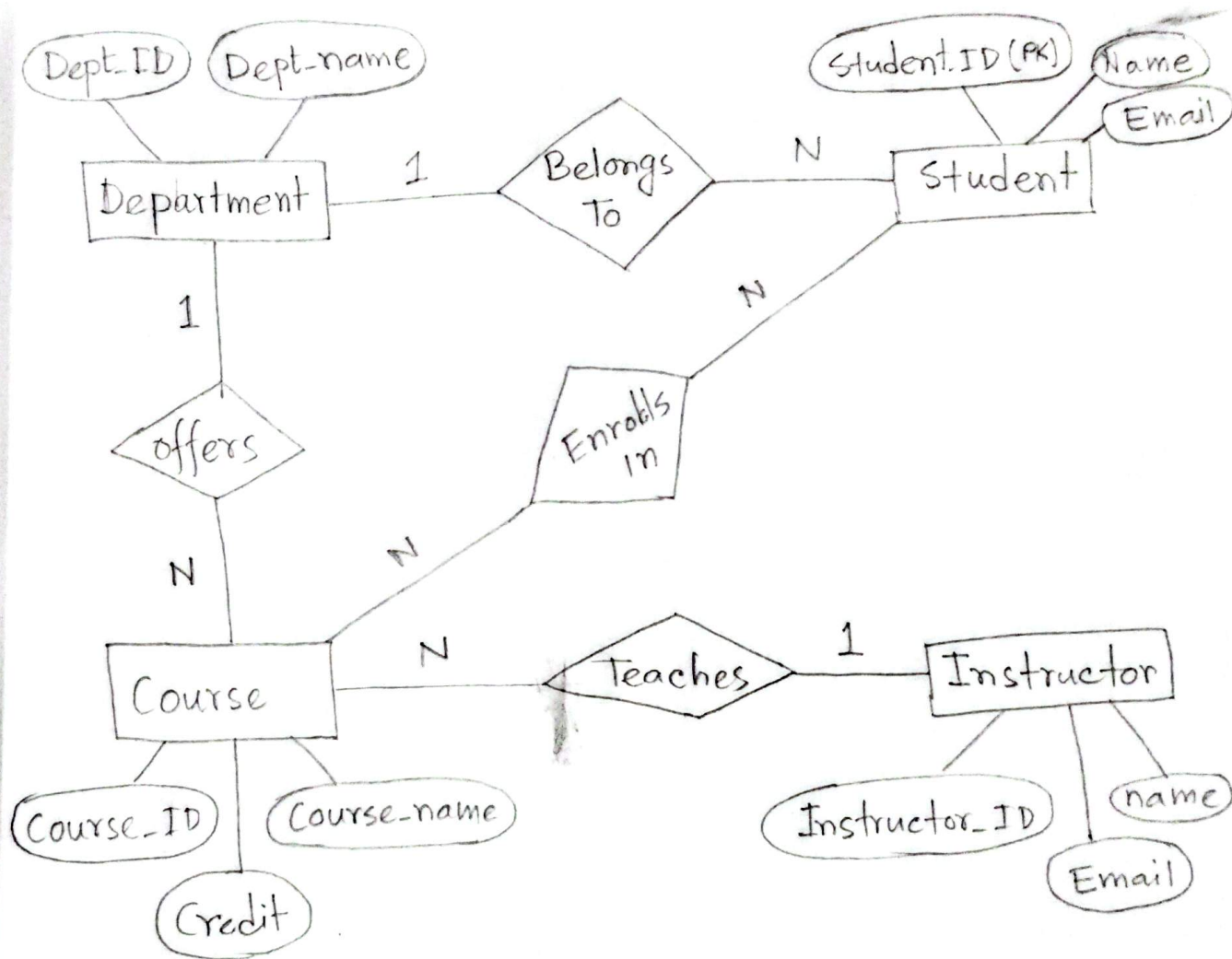
```
CREATE TABLE student (
    ID INT PRIMARY KEY,
    Name VARCHAR(20) NOT NULL,
    Email VARCHAR(50) UNIQUE,
    Age INT CHECK (Age >= 19)
);
```

2) a) Entities and attribute:

- i) Student → student-ID (PK), Name, Email
- ii) Department → Dept-ID (PK), Dept name
- iii) Course → Course-ID (PK), Course-name, Credit
- iv) Instructor → Instructor-ID (PK), Name, Email

Relationship and Cardinality:

- i) Belongs To → student (N) to Department (1)
- ii) Offers → Department (1) to course (N)
- iii) Teaches → Instructor (1) to course (N)
- iv) Enrolls in → student (N) to Course (N)



3) An SQL create table statement satisfying the given requirements is constructed below:

```
CREATE TABLE instructor(  
    ID INT PRIMARY KEY,  
    Name VARCHAR(20) NOT NULL,  
    dept-name VARCHAR(30),  
    salary INT,  
    FOREIGN KEY (dept-name) REFERENCES  
        department(dept-name)  
);
```

4)

a) SELECT name, dept
FROM instructor;

b) SELECT DISTINCT dept
FROM instructor;

c) SELECT ID, name, (salary/12) AS monthly-salary
FROM instructor;

d) SELECT name
FROM instructor
WHERE dept = 'cs' AND salary > 70000;

- e) SELECT *
FROM instructor
WHERE dept = 'CS' OR dept = 'Physics';
- f) SELECT *
FROM instructor
WHERE name LIKE '%i%';
- g) SELECT *
FROM instructor
WHERE name LIKE '-----';
- h) SELECT *
FROM instructor
ORDER BY salary DESC, name ASC;
- i) SELECT *
FROM instructor
WHERE salary BETWEEN 70000 AND 90000;
- j) SELECT *
FROM instructor
WHERE (ID, dept) = (102, 'CS');

k) (SELECT *
FROM instructor
WHERE salary > 80000)
UNION
(SELECT *
FROM instructor
WHERE dept = 'Physics');

l) (SELECT *
FROM instructor
WHERE salary > 70000)
INTERSECT
(SELECT *
FROM instructor
WHERE dept = 'Physics');

m) (SELECT *
FROM instructor
WHERE salary > 70000)
EXCEPT
(SELECT *
FROM instructor
WHERE dept = 'Physics');

n) (SELECT *
FROM instructor
WHERE dept = 'CS')
UNION ALL
(SELECT *
FROM instructor
WHERE salary > 75000);

o) i) SELECT *
FROM instructor
WHERE salary IS NULL;
ii) SELECT *
FROM instructor
WHERE salary IS NOT NULL;

p) SELECT AVG(salary), MIN(salary), MAX(salary),
SUM(salary), COUNT(salary)
FROM instructor;

q) SELECT dept, AVG(salary)
FROM instructor
GROUP BY dept;

r) `SELECT dept, AVG(salary)`
`FROM instructor`
`GROUP BY dept`
`HAVING AVG(salary) > 70000;`

s) `SELECT *`
`FROM instructor`
`WHERE salary > (SELECT AVG(salary)`
`FROM instructor);`

t) `SELECT *`
`FROM instructor`
`WHERE salary = (SELECT MAX(salary)`
`FROM instructor);`

u)

```
String url = "jdbc:mysql://localhost:5000/university";  
String user = 'root';  
String password = "password";  
Connection conn = DriverManager.getConnection(  
    url, user, password);  
Statement stmt = conn.createStatement();
```

v) try {

```
string updateSql = " UPDATE instructor  
                    SET salary = 85000  
                    WHERE ID = 101 ";
```

```
int rowsAffected = stmt.executeUpdate(updateSql);
```

```
system.out.println("Rows updated: " + rowsAffected);
```

```
} catch (SQLException e) {
```

```
    System.out.println("SQL ERROR: " + e.getMessage());
```

```
}
```

w) string selectSql = " SELECT ID, name
 FROM instructor ";

```
ResultSet rset = stmt.executeQuery(selectSql);
```

```
while (rset.next()) {
```

```
    System.out.println("ID: " + rset.getInt("ID") +
```

```
    ", Name: " + rset.getString("name"));
```

5) a) CREATE TABLE department (
dept-name VARCHAR(50) PRIMARY KEY,
building VARCHAR(100)
);

CREATE TABLE instructor (
ID INT PRIMARY KEY,
name VARCHAR(30),
dept-name VARCHAR(50),
salary INT,
FOREIGN KEY (dept name) REFERENCES
department (dept-name)
);

b) SQL query:

SELECT *
FROM instructor, department;

ID	Name	dept	salary	dept_name	building
101	Smith	CS	60000	CS	Taylor
101	Smith	CS	60000	Biology	Watson
101	Smith	CS	60000	Finance	Painter
101	Smith	CS	60000	Physics	Watson
102	KIM	CS	80000	CS	Taylor
102	Kim	CS	80000	Biology	Watson
102	Kim	CS	80000	Finance	Painter
102	Kim	CS	80000	Physics	Watson
103	Wu	Finance	90000	CS	Taylor
103	Wu	Finance	90000	Biology	Watson
103	Wu	Finance	90000	Finance	Painter
103	Wu	Finance	90000	Physics	Watson
104	Gold	Physics	70000	CS	Taylor
104	Gold	Physics	70000	Biology	Watson
104	Gold	Physics	70000	Finance	Painter
104	Gold	Physics	70000	Physics	Watson
105	Darwin	Physics	NULL	CS	Taylor
105	Darwin	Physics	NULL	Biology	Watson
105	Darwin	Physics	NULL	Finance	Painter
105	Darwin	Physics	NULL	Physics	Watson
106	Crick	Biology	75000	CS	Taylor
106	Crick	Biology	75000	Biology	Watson
106	Crick	Biology	75000	Finance	Painter
106	Crick	Biology	75000	Physics	Watson